Introduction:

The word polymorphism is derived from two Greek words: "poly", meaning many, and "morph", meaning form. Thus, polymorphism refers to the ability of an entity to exist or operate in more than one form.

For example:
The + operator in C++ is used to perform two specific functions. When it is used with numbers (integers and floating-point numbers), it performs addition.

```
int a = 5;
int b = 6;
int sum = a + b;    // sum = 11
```

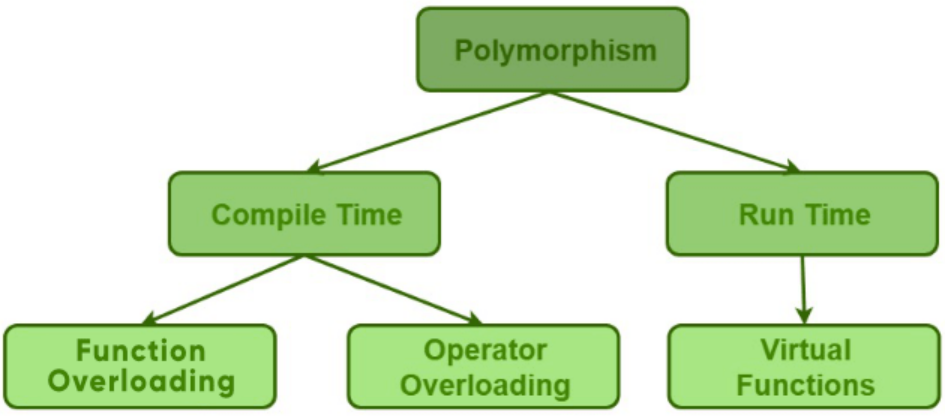
And when we use the + operator with strings, it performs string concatenation. For example,

```
string firstName = "abc ";
string lastName = "xyz";

// name = "abc xyz"
string name = firstName + lastName;
```

Types:

1. Compile Time Polymorphism (Static Binding)
2. Run Time Polymorphism (Dynamic Binding)



1. **Compile Time Polymorphism:** Compile-time polymorphism occurs when the function to be executed is determined at compile time.

   It is further categorized into two types:

a. **Function Overloading:**

When multiple functions in a class with the same name but different parameters exist, these functions are said to be overloaded.

Why Use Function Overloading?

- To improve readability and reusability of code.
- To perform similar operations with different types or different numbers of arguments using a single function name.

Example:

main.cpp

Share Run

Output

```cpp
#include <iostream>
using namespace std;

class Calculator {
public:
    // Function to add two integers
    int add(int a, int b) {
        return a + b;
    }

    // Function to add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Function to add two doubles
    double add(double a, double b) {
        return a + b;
    }
};

int main() {
    Calculator calc;

    cout << "Sum of 2 and 3: " << calc.add(2, 3) << endl;
    cout << "Sum of 1, 2 and 3: " << calc.add(1, 2, 3) << endl;
    cout << "Sum of 2.5 and 3.5: " << calc.add(2.5, 3.5) << endl;

    return 0;
}
```

```
Sum of 2 and 3: 5
Sum of 1, 2 and 3: 6
Sum of 2.5 and 3.5: 6


=== Code Execution Successful ===
```

b. **Operator Overloading:** This topic is thoroughly discussed in Unit-3. Please refer to that chapter for a detailed explanation and in-depth examples related to operator overloading.

2. **Run Time Polymorphism:** Run-Time Polymorphism, also known as Dynamic Binding, occurs when the decision about which function to call is made at runtime, rather than at compile time.
   a. **Method Overriding using Virtual function:**

      Method overriding refers to the process of creating a new definition of a function in a derived class that is already defined inside its base class.

      Example:

We will learn about virtual function later in this chapter.

New and Delete operator:

In C++, new and delete are operators used to allocate and deallocate memory dynamically on the heap during runtime.

Using new :

Syntax:

```
data_type* pointer = new data_type;          // For single variable

data_type* array = new data_type[size];      // For array
```

Using delete:

Syntax:

```
delete pointer;         // For single variable

delete[] array;         // For array
```

Example:

main.cpp

```cpp
#include <iostream>
using namespace std;
class Animal {
public:
    Animal() {
        cout << "Animal constructor called\n";
    }
    virtual void sound() {
        cout << "Animal makes a sound\n";
    }
    virtual ~Animal() {
        cout << "Animal destructor called\n";
    }
};
class Dog : public Animal {
public:
    Dog() {
        cout << "Dog constructor called\n";
    }
    void sound() override {
        cout << "Dog barks\n";
    }
    ~Dog() {
        cout << "Dog destructor called\n";
    }
};
int main() {
    Animal* a = new Dog();  // Base class pointer to derived class object
    a->sound();             // Calls Dog's version due to virtual function
    delete a;               // Calls Dog and Animal destructors (polymorphic delete)
    return 0;
}
```

Output

```
Animal constructor called
Dog constructor called
Dog barks
Dog destructor called
Animal destructor called


=== Code Execution Successful ===
```

Pointer to an Object:

A pointer to an object is a pointer that holds the address of a class object. This concept becomes especially powerful when used with inheritance and virtual functions, enabling runtime polymorphism.

Syntax:

```
ClassName* ptr;

ptr = new ClassName();
```

Example:

```cpp
class Animal {
public:
    virtual void sound() {
        cout << "Animal makes a sound." << endl;
    }

    virtual ~Animal() {
        cout << "Animal destructor called." << endl;
    }
};
class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks." << endl;
    }
    ~Dog() {
        cout << "Dog destructor called." << endl;
    }
};
int main() {
    Animal* a = new Dog();   // Runtime polymorphism
    a->sound();
    delete a;
    return 0;
}
```

```
Dog barks.
Dog destructor called.
Animal destructor called.


=== Code Execution Successful ===
```

Pointer to derived class:

A pointer to a derived class is a pointer that points to an object of the derived class. Unlike base class pointers, it can access both base and derived class members (as long as they are public or protected).

Example:

main.cpp | Share | Run | Output

```cpp
#include <iostream>
using namespace std;
class Animal {
public:
    void eat() {
        cout << "Animal eats food." << endl;
    }
};
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog barks." << endl;
    }
};
int main() {
    Dog d;
    // Pointer to derived class
    Dog* ptr = &d;
    ptr->eat();   // Inherited from Animal
    ptr->bark();  // Own function of Dog
}

```

```
Animal eats food.
Dog barks.


=== Code Execution Successful ===
```

Virtual Functions:

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. It is declared using the virtual keyword.

Syntax:

```
class Base{

public:

    virtual return_type functionName(){

        // Function body

    }

}

class Derived : public Base{

public:

    return_type functionName() override {

        // Function body

    }

}
```

Example: "The example demonstrating method overriding with virtual functions is given earlier in this chapter."

Friend Function:

A friend function is a function that is not a member of a class but is granted access to its private and protected members. It cannot be called using class objects (since it's not a member).

Syntax:

```
class ClassName {

    friend return_type function_name(parameters);
```

```
};
```

Example:

Friend class:

A friend class is a class that is granted access to the private and protected members of another class.

Syntax:

```
class ClassB; // Forward declaration

class ClassA {

    friend class ClassB;  // ClassB is a friend of ClassA

private:

    int secretData;

public:

    ClassA() : secretData(12345) {}

};
```

Real-Life Example: Doctor and MedicalRecord

A Doctor needs to view a patient's private medical record, but MedicalRecord class keeps its data private. To allow the Doctor class to access the data, we make it a friend of the MedicalRecord class.

Example:

main.cpp

Share Run Output

```cpp
#include <iostream>
using namespace std;
// forward declaration
class ClassB;
class ClassA {
    private:
        int numA;
        friend class ClassB;
    public:
        // constructor to initialize numA to 12
        ClassA() : numA(12) {}
};
class ClassB {
    private:
        int numB;
    public:
        // constructor to initialize numB to 1
        ClassB() : numB(1) {}
    // member function to add numA
    // from ClassA and numB from ClassB
    int add() {
        ClassA objectA;
        return objectA.numA + numB;
    }
};
int main() {
    ClassB objectB;
    cout << "Sum: " << objectB.add();
    return 0;
}
```

```
Sum: 13

=== Code Execution Successful ===
```

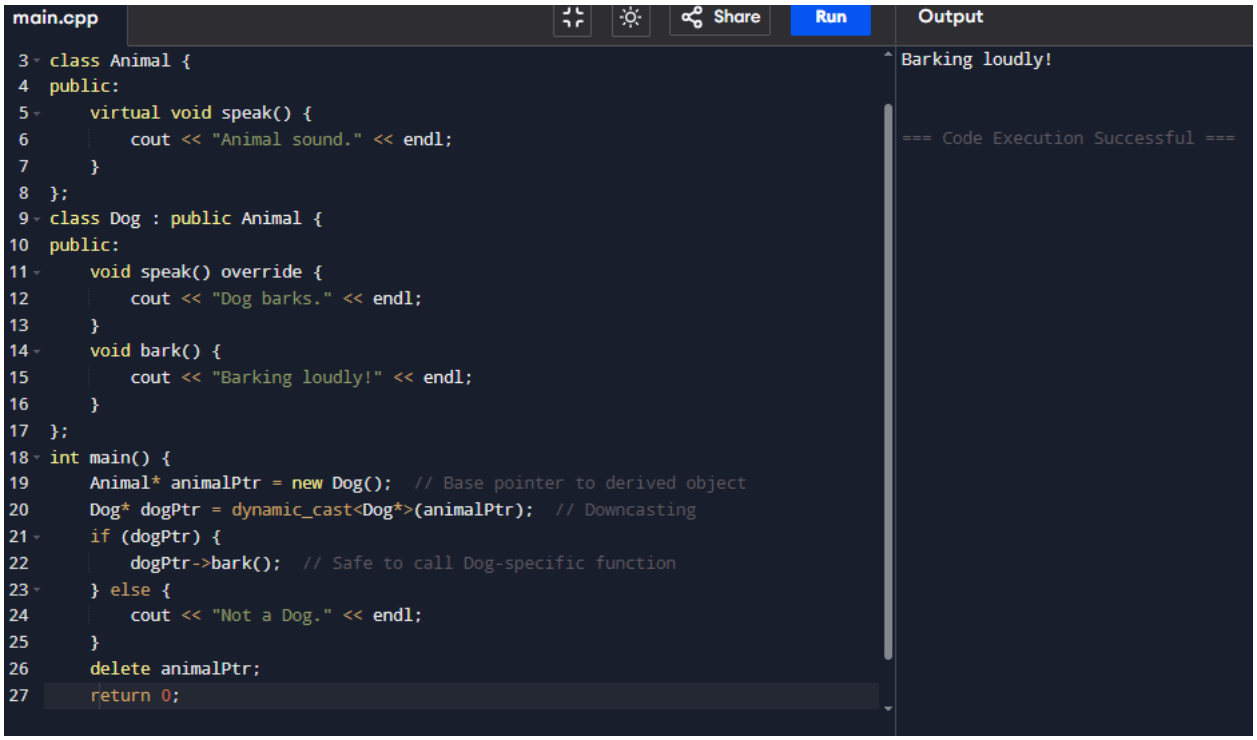
Dynamic cast operator:

In C++, dynamic_cast is a cast operator that converts data from one type to another type at runtime. It is mainly used in inherited class hierarchies for safely casting the base class pointer or reference to derived class (called downcasting). To work with dynamic_cast, there must be one virtual function in the base class.

Syntax

```
dynamic_cast <new_type>(expression)
```

Example:



Typed Operator:

The term "typed operator" generally refers to C++ type casting operators that convert data from one type to another. These operators are explicit type converters that make code more readable, safe, and efficient.

C++ provides four specific type casting operators, each with its purpose:

a. **static_cast<>:** Used for standard type conversions.
   - Performs checks at compile-time.
   - Converts between related types like:
     int to float

     base class to derived class (if safe)

Syntax:

static_cast<target_type>(expression)

Example:

int a = 10;

double b = static_cast<double>(a); // Converts int to double

b. **Dynamic_cast<>:** Already discussed.

c. **const_cast<>:** Used to add or remove const/volatile qualifiers.
   - Allows modification of values that were originally declared as const.

Syntax:

const_cast<target_type>(expression)

Example:

main.cpp — Share — Run — Output

```cpp
#include <iostream>
using namespace std;

void print(int* ptr) {
    *ptr = 50; // Modify the value
    cout << "Modified value: " << *ptr << endl;
}

int main() {
    const int x = 10;

    // Remove constness to pass to a function expecting non-const
        pointer
    print(const_cast<int*>(&x));

    return 0;
}
```

```
Modified value: 50


=== Code Execution Successful ===
```

d. **reinterpret_cast<>:**Used for low-level casting between unrelated types.
   - Does not guarantee safety or meaningful conversion.
   - Only preserves bit patterns.

Syntax:

reinterpret_cast<target_type>(expression)

Example:

main.cpp — Share — Run — Output

```cpp
#include <iostream>
using namespace std;

int main() {
    int num = 65;

    // Convert int* to char* using reinterpret_cast
    char* ptr = reinterpret_cast<char*>(&num);

    cout << "Character representation: " << *ptr << endl;

    return 0;
}
```

```
Character representation: A


=== Code Execution Successful ===
```

Summary Table:

Cast Type	Use Case	Safety	Type of Check
static_cast	Standard conversions, upcasting	Safe	Compile-time
dynamic_cast	Downcasting in polymorphism	Safe	Runtime
const_cast	Add/remove const or volatile	Risky	No check
reinterpret_cast	Bit-level reinterpreting of types	Dangerous	No check

****Thank You****